National University of Computer and Emerging Sciences



Lab Manual 05
Artificial Intelligence

Course Instructor	Ms. Bushra Rashid
Lab Instructor	Muhammad Hasan
Lab Instructor Email ID	m.hasan@lhr.nu.edu.pk
Semester	Fall 2026

FAST - School of Computing
FAST-NU, Lahore Campus

Contents

Lab Manual 5 4
Objectives: 4
Activities: 4
Activity 1 (Python): 4
File Handling 4
Writing to a csv file 4
Reading from a csv file 4
Writing to a json file 4
Reading from a json file 4
Working on csv file 5
Reading 5
Deleting the Header Row having column names 5
sum of each row 5
sum of each row with built-in sum function and list comprehension 5
Loading Google Drive 6
Try and Except Blocks 6
Simple Exception 6
Multiple Exception 6
else Block 6
finally Block 7
'finally' without 'except' 7
Raising Exception 7
Raising build-in exception 7
Example: 8
Custom Exceptions 8
Simple Custom Exception 8
Simple Custom Exception with super() 8
Exercise 1: Count the number of words in a text file 9
Exercise 2: Read a CSV and find the average of a column 9
Exercise 3: Implement code to handle errors during numerical input 9
Exercise 4: Implement a code to handle exception for invalid file formats - txt 9
OOP in Python 10
Advanced OOP Concepts - Polymorphism 10
Example 10
Advanced OOP Concepts - Encapsulation 12

Getters & Setters Functions 12
Example 13
Advanced OOP Concepts - Abstraction 13
Example 14
Exercise: Implement a banking system with classes Bank, Customer and Account . 15
Modules 16
#Example 16
importing my_first_module.py 17
Packages 17
Importing Modules & Functions 17
Activity 2 (Informed Search Algorithms): 18
Informed Search Algorithms 18
What is a Heuristic? 18
Properties of a Good Heuristic: 18
Heuristic Function in Python 18
Types of Informed Search Algorithms 18
Greedy Best-First Search (GBFS) 18
Evaluation Function: 18
Algorithm 19
Properties 19
A* Search Algorithm 19
Evaluation Function: 19
Algorithm 19
Properties 19
Comparison Summary 19
Game Playing in Artificial Intelligence 20
Characteristics of Game Playing Problems 20
Game State Representation 20
Game Tree 20
Utility (Evaluation) Function 20
Activity 3 (Problem Statements / Tasks): 20
1. CSV Student Performance Analyzer 20
2. Inventory Management System 21
3. Implement Greedy and A* Search 22

Lab Manual 5

Objectives:

- ✓ File Handling
  csv & json files, Exception Handling
- ✓ OOP in Python
  Polymorphism, Encapsulation, Abstraction, Modules & Packages
- ✓ Informed Search Algorithms
  Greedy Best-First Search (GBFS), A* Search Algorithm, Minimax Algorithm

Activities:

Activity 1 (Python):

File Handling

```
# Writing to a csv file
import csv

data =[["Name", "Age"], ["Alice", 30], ["Bob", 25]]
with open("data.csv", "w", newline="") as file:
  writer = csv.writer(file)
  writer.writerows(data)

# Reading from a csv file
import csv

with open("data.csv", "r") as file:
  reader = csv.reader(file)
  for row in reader:
    print(row)

# Writing to a json file
import json

data = {"Name": "Alice", "Age": 30}
with open("data.json", "w") as file:
  json.dump(data, file, indent=4)

data_json = [{'name': 'Ali', 'age': 30, 'student': 'yes'},
                  {'name': 'Ahmad', 'age': 40, 'student': 'no'}]
with open('output.json', 'w') as file:
  json.dump(data_json, file, indent=4)

# Reading from a json file
import json
with open("data.json", "r") as file:
  data = json.load(file)
  print(data)
```

```
# Working on csv file
# Reading
import csv

with open("seeds.csv", "r") as file:
  reader = csv.reader(file)

  for r in reader:
    print(r)

# Deleting the Header Row having column names
import csv

with open("seeds.csv", "r") as file:
  reader = csv.reader(file)

  my_data = []
  for r in reader:
    my_data.append(r)

del my_data[0]
print(my_data)

# sum of each row
import csv

with open("seeds.csv", "r") as file:
  reader = csv.reader(file)

  my_data = []
  for r in reader:
    my_data.append(r)

del my_data[0]
print(my_data)

for row in my_data:
  sum_of_row = 0
  for element in row:
    sum_of_row += float(element)
  print(sum_of_row)
# sum of each row with built-in sum function and list comprehension
import csv

with open("seeds.csv", "r") as file:
  reader = csv.reader(file)

  my_data = []
```

```
  for r in reader:
    my_data.append(r)

del my_data[0]
print(my_data)

for row in my_data:
  row = [float(item) for item in row]
  sum_of_row = sum(row)
  print(sum_of_row)
```

Loading Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

Try and Except Blocks

```
# Execution stops due to error
# That's why we use try except blocks

# execution starts from 'try' block
# if 'try' executes successfully, 'else' also executes
# if 'try' block has an error, 'except' block executes
# 'finally' block always gets executed
```

Simple Exception

```
try:
  num = int(input("Enter a number: "))
  result = 10 / num
  print("Result:", result)
except ZeroDivisionError:
  print("Error: Division by zero is not allowed!")
except ValueError:
  print("Error: Invalid input. Please enter a valid number.")
```

Multiple Exception

```
try:
  num = int(input("Enter a number: "))
  result = 10 / num
  print("Result:", result)
except (ZeroDivisionError, ValueError) as e:
  print("An error occurred:", e)
```

else Block

```
try:
  num = int(input("Enter a number: "))
  result = 10 / num
```

```
  print("Result:", result)
except (ZeroDivisionError, ValueError) as e:
  print("An error occurred:", e)
else:
  print("Else Block")
```

finally Block

```
try:
  file = open("example.txt", "r")
  content = file.read()
  print(content)
except FileNotFoundError:
  print("Error: The file does not exist.")
finally:
  file.close()
  print("File closed successfully.")
```

'finally' without 'except'

```
try:
  print("trying to divide by zero.")
  result = 10 / 0
finally:
  print("This will always run, regardless of an error")
```

Raising Exception

```
def check_positive(num):
  if num < 0:
    raise ValueError("Negative numbers are not allowed.")
  return num

try:
  print(check_positive(5))
except ValueError as e:
  print("Error:", e)
```

Raising build-in exception

```
def divide(a, b):
  if b == 0:
    raise ZeroDivisionError("Cannot divide by zero!")
  return a / b
try:
  result = divide(10, 0)
  print("Result:", result)
except ZeroDivisionError as e:
  print("Error:", e)
```

```
# Example:
def check_age(age):
  if age <= 0:
    raise ValueError('Age is either zero or negative')
  else:
    print('You have entered correct age.')

try:
  check_age(5)
except ValueError as e:
  print(e)
```

Custom Exceptions

```
# Simple Custom Exception
class InvalidAgeError(Exception):
  """Exception raised for invalid age."""
  pass

def check_age(age):
  if age < 0:
    raise InvalidAgeError("Age cannot be negative!")
  print(f"Age is valid: {age}")

try:
  check_age(25)
  check_age(-5)
except InvalidAgeError as e:
  print("Error:", e)
```

```
# Simple Custom Exception with super()
class WithdrawalError(Exception):
  """Raise Error if the withdraw amount is greater than balance"""
  def __init__(self, balance, amount):
    self.balance = balance
    self.amount = amount
    super().__init__(f"Insufficient balance! Tried to withdraw
{self.amount}, but only {self.balance} is available")

def withdraw(balance, amount):
  if amount > balance:
    raise WithdrawalError(balance, amount)
  return balance - amount
try:
  print("New Balance:", withdraw(100, 50))
  print("New Balance:", withdraw(100, 150))
except WithdrawalError as e:
  print("Error:", e)
```

```python
# Exercise 1: Count the number of words in a text file
with open("sample.txt", "r") as file:
  content = file.read()
  words = content.split()
  print(f'Number of words in the text file are: {len(words)}')

# Exercise 2: Read a CSV and find the average of a column
import csv

with open ('./sample_data/california_housing_test.csv') as file:
  reader = csv.DictReader(file)

  sum = 0
  count = 0

  for row in reader:
    sum += float(row['housing_median_age'])
    count += 1

  print(f'Average of the column "housing_median_age" is {sum/count}')

# Exercise 3: Implement code to handle errors during numerical input
class RangeError(Exception):
  """Raise Error if the input is out of range"""
  pass

def get_input():
  num = input('Please enter a number between 0 and 100: ')

  num = float(num)

  if num < 0 or num > 100:
    raise RangeError("Number is less than 0 or greater than 100")

  return num

while True:
  try:
    num = get_input()
    break
  except (RangeError, ValueError) as e:
    print(e)
# Exercise 4: Implement a code to handle exception for invalid file formats - txt
class InvalidFileFormatError(Exception):
  """
  Exception to be raised if input file format is not a txt
  """
  def __init__(self, fn):
    self.filename = fn
```

```
    super().__init__(f'Invalid file format, only txt is supported')

try:
  path_file = "sample.txt"

  if not path_file.endswith('.txt'):
    raise InvalidFileFormatError(path_file)

  print('File format is correct.')
except InvalidFileFormatError as e:
  print(e)
```

OOP in Python

```
# Advanced OOP Concepts - Polymorphism
class Animal:   # Parent Class
  def speak(self):
    return "Some generic animal sound"

class Dog (Animal):   # Child Class
  def speak(self):    # Overriding speak()
    return "Bark"

class Cat (Animal):   # Child Class
  def speak(self):    # Overriding speak()
    return "Meow"

# creating objects
dog = Dog()
cat = Cat()

print(dog.speak())
print(cat.speak())
```

Example

```
# Base Class
class Animal:
    def __init__(self, name):
        self.name = name
        print(f"Animal Constructor Called for {self.name}")

    def eat(self):
        print(f"{self.name} is eating.")

    def speak(self):
      print('Generic Sound')

# Derived Class from Animal
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)  # Call parent constructor
```

```
        self.breed = breed
        print(f"Dog Constructor Called for {self.name}")

    def speak(self):
      print('Dog is barking.')

# Derived Class from Animal
class Fish(Animal):
    def __init__(self, name, water_type):
        Animal.__init__(self, name)
        self.water_type = water_type
        print(f"Fish Constructor Called for {self.name}")

    def swim(self):
        print(f"{self.name} is swimming in {self.water_type} water.")

    def speak(self):
      print('Fish are generally silent')

# Multilevel Inheritance (Dolphin -> Fish -> Animal)
class Dolphin(Fish):
    def __init__(self, name, water_type, intelligence_level):
        super().__init__(name, water_type)
        self.intelligence_level = intelligence_level
        print(f"Dolphin Constructor Called for {self.name}")

    def jump(self):
        print(f"{self.name} is jumping out of the water!")

    def speak(self):
      print('Dolphins are whistling')

# Creating Objects
print("----- Animal Object -----")
animal1 = Animal("Generic Animal")

print("\n----- Dog Object -----")
dog1 = Dog("Buddy", "Golden Retriever")
print("\n----- Fish Object -----")
fish1 = Fish("Nemo", "Fresh")

print("\n----- Dolphin Object -----")
dolphin1 = Dolphin("Flipper", "Salt", "High")

# Calling functions for each object
print("\n----- Animal Object Function -----")
animal1.speak()

print("\n----- Dog Object Function -----")
dog1.speak()
```

```
print("\n----- Fish Object Functions -----")
fish1.speak()

print("\n----- Dolphin Object Functions -----")
dolphin1.speak()
```

Advanced OOP Concepts - Encapsulation

```
# public_var
# _protected_var - single underscore is just the convention,
# it does not stop being access outside the classes
# __private_var

class BankAccount:
  def __init__(self, owner, balance):
    self.owner = owner           # public variable
    self.__balance = balance     # private variable

# Creating an object
account = BankAccount("Alice", 1000)

# Trying to access the private attribute
print(account.owner)         # works fine
print(account.__balance)     # AttributeError
```

Getters & Setters Functions

```
class BankAccount:
  def __init__(self, owner, balance):
    self.owner = owner           # public variable
    self.__balance = balance     # private variable

  def get_balance(self):         # Getter Method
    return self.__balance

  def set_balance(self, amount):         # Setter Method
    if amount < 0:
      print("Balance cannot be negative!")
    else:
      self.__balance = amount

# Creating an object
account = BankAccount("Alice", 1000)

# Accessing balance using getter
print(account.get_balance())

# Updating balance using setter
account.set_balance(1500)
```

```python
# Again accessing balance using getter
print(account.get_balance())

# Example
class BankAccount:
  def __init__(self, name, bal):
    self.name = name
    self.__balance = bal

  def deposit(self, amt):
    self.__balance += amt

  def print_bal(self):
    print(self.__balance)     # Works fine

  def get_balance(self):
    return self.__balance

  def set_balance(self, amount):
    if self.__balance == 0:
      if amount < 0:
        print("Balance cannot be negative.")
      else:
        self.__balance += amount
    else:
      print("Balance already set.")

class SavingAccount(BankAccount):
  def __init__(self, name, bal):
    super().__init__(name, bal)
    self.acc_type = "Savings"

  def print_bal(self):
    print(self.__balance)      # AttributeError

b1 = BankAccount('Ali', 500)
b1.print_bal()
b1.set_balance(100)
print(b1.get_balance())

b2 = SavingAccount('Ahmad', 500)
b2.print_bal()
```

Advanced OOP Concepts - Abstraction

```python
from abc import ABC, abstractmethod
# Abstract Class
class Vehicle(ABC):
  @abstractmethod
  def start(self):
    pass   # No implementation, subclasses must define this method
```

```
  @abstractmethod
  def stop(self):
    pass   # No implementation, subclasses must define this method

# Concrete Class (inherits from abstract class)
class car(Vehicle):
  def start(self):
    print("Car is starting...")

  def stop(self):
    print("Car is stoping...")

my_car = car()      # concrete class can be instantiated while abstract
class can't
my_car.start()
my_car.stop()
```

Example

```
from abc import ABC, abstractmethod

# Abstract Class
class BankAccount(ABC):
  def __init__(self, name, balance):
    self.name = name
    self.balance = balance

  @abstractmethod
  def deposit(self, amount):
    pass        # No implementation, subclasses must define this method
  @abstractmethod
  def withdraw(self, amount):
    pass        # No implementation, subclasses must define this method

# Concrete Class
class SavingAccount(BankAccount):
  def deposit(self, amount):
    if amount < 0:
      print("Cannot deposit negative amount.")
    else:
      self.balance += amount
      print(f"Deposited ${amount}. New Balance: ${self.balance}")

  def withdraw(self, amount):
    if amount < 0:
      print ("Cannot withdraw negative amount.")
    elif amount > self.balance:
      print("Insufficient Funds!")
    else:
      self.balance -= amount
      print(f"Withdrawn ${amount}. New Balance: ${self.balance}")
```

```python
# Creating an object
account = SavingAccount('Ali', 1000)
account.deposit(500)
account.withdraw(200)
account.withdraw(2000)

# Exercise: Implement a banking system with classes Bank, Customer and Account
class Account:
    def __init__(self, balance=0):
        self.__balance = balance

    def get_balance(self):
        return self.__balance

    def deposit(self, amt):
        self.__balance += amt

    def withdraw(self, amt):
        if self.__balance >= amt:
            self.__balance -= amt
            print(f'Remaining Balance: {self.__balance}')
        else:
            print('Insufficient funds')

class Customer:
    def __init__(self, id, name, add):
        self.__id = id
        self.name = name
        self.address = add
        self.account = Account()

    def get_name(self):
        return self.name

    def get_address(self):
        return self.address

    def set_address(self, add):
        self.address = add
        print(f'Address updated to {add}')

    def deposit(self, amt):
        self.account.deposit(amt)

    def withdraw(self, amt):
        self.account.withdraw(amt)

    def get_balance(self):
        return self.account.get_balance()
```

```
class Bank:
    def __init__(self, bn):
        self.bank_name = bn
        self.customers = {}

    def add_customer(self, id, n, add):
        self.customers[id] = Customer(id, n, add)

    def get_customer(self, id):
        return self.customers.get(id, 'None')

bank = Bank('ABC Bank')

bank.add_customer(1, 'Ben', 'XYZ Street')

current_customer = bank.get_customer(1)

current_customer.deposit(200)
print(current_customer.get_balance())

current_customer.deposit(100)
print(current_customer.get_balance())
current_customer.withdraw(200)
print(current_customer.get_balance())

current_customer.withdraw(150)
print(current_customer.get_balance())
```

Modules

#Example

```
%%writefile my_first_module.py

def add(a, b):
  return a + b

def subtract(a, b):
  return a - b

def frequency_unique(df_col):
  '''
  This function returns the frequency of unique values in a python dict format
  '''
  val_counts = df_col.value_counts().reset_index()

  output_dict = {float(row[1][0]): int(row[1][1]) for row in
val_counts.iterrows()}

  return output_dict
```

```
# importing my_first_module.py
import my_first_module
import pandas as pd

my_first_module.add(4, 5)

df = pd.read_csv('sample_data/california_housing_test.csv')

my_first_module.frequency_unique(df['housing_median_age'])

# Packages
# Many modules in one package
!mkdir my_package

# Next Cell
%%writefile my_package/__init__.py

# Next Cell
%%writefile my_package/math_operations.py
def add(a, b):
  return a + b

def subtract(a, b):
  return a - b

def frequency_unique(df_col):
  '''
  This function returns the frequency of unique values in a python dict
format
  '''
  val_counts = df_col.value_counts().reset_index()

  output_dict = {float(row[1][0]): int(row[1][1])
                       for row in val_counts.iterrows()}
  return output_dict

# Next Cell
%%writefile my_package/greeting_operations.py

def greet(name):
  print(f'Hello {name}, welcome to code')

def bye(name):
  print(f'Good Bye {name}, We hope you enjoyed our program.')

# Importing Modules & Functions
from my_package import  math_operations
result = math_operations.subtract(3, 1)
print(result)
```

```
from math import *
print(cos(0))

import pandas as pd
data = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})
print(data)
```

Activity 2 (Informed Search Algorithms):

Informed Search Algorithms

Informed search algorithms (also called heuristic search algorithms) use additional problem-specific knowledge to guide the search toward the goal more efficiently.
Unlike blind search algorithms (BFS, DFS, UCS), informed search algorithms use a heuristic function to estimate how close a state is to the goal.
They make use of:

- Initial state
- Goal state
- Actions available between states
- Heuristic information

What is a Heuristic?

A heuristic function, denoted as h(n), estimates the cost from node n to the goal.
It does not give the exact cost — it gives an estimate.

Properties of a Good Heuristic:

- Problem-specific
- Improves search efficiency
- Should not overestimate (for optimality in A*)

Heuristic Function in Python

A heuristic is typically implemented as a dictionary or a function:

```
heuristic = {
    'A': 6,
    'B': 4,
    'C': 2,
    'D': 3,
    'E': 1,
    'G': 0
}
```

or

```
def h(state):
    return heuristic[state]
```

Types of Informed Search Algorithms

Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the node that appears closest to the goal.

Evaluation Function:

$$f(n) = h(n)$$

It ignores the path cost and uses only heuristic value.

Algorithm

- Create priority queue ordered by h(n)
- Insert start node
- While queue not empty:
  - Remove node with smallest h(n)
  - If goal → return solution
  - Expand neighbors
  - Insert unexplored neighbors

Properties

Property	*Value*
Optimal	No
Complete	Not always
Memory	Low
Speed	Fast

A Search Algorithm*

A* combines:

- Uniform Cost Search (path cost)
- Greedy Best-First Search (heuristic)

Evaluation Function:

$$f(n) = g(n) + h(n)$$

Where:

- g(n) → Cost from start to n
- h(n) → Estimated cost from n to goal

Algorithm

- Insert start node with cost 0
- While queue not empty:
  - Remove node with smallest f(n)
  - If goal → return
  - Expand neighbors
  - Compute new g(n), h(n), f(n)
  - Update queue if better path found

Properties

Property	*Value*
Optimal	No
Complete	Not always
Memory	Low
Speed	Fast

Comparison Summary

Algorithm	*Evaluation*	*Optimal*	*Complete*
GBFS	h(n)	No	No
*A**	g(n) + h(n)	Yes	Yes

Game Playing in Artificial Intelligence

Game playing is one of the classical problems in Artificial Intelligence. In game-playing problems, an intelligent agent must make decisions by considering possible future moves and the responses of an opponent.

These problems are usually modelled as:

- Two-player
- Turn-based
- Zero-sum games

Examples include:

- Tic-Tac-Toe
- Chess
- Checkers
- Connect Four

Characteristics of Game Playing Problems

- There are two players:
  - MAX (the AI or computer)
  - MIN (the opponent)
- Players take turns
- One player's gain is the other player's loss
- Both players are assumed to play optimally

Game State Representation

In game playing:

- A state represents a configuration of the game board
- The initial state is the starting position
- Terminal states are game-ending positions (win, lose, or draw)
- Each state has possible actions leading to new states
- The game is represented as a game tree.

Game Tree

A game tree is a tree structure where:

- Each node represents a game state
- Each edge represents a possible move
- Levels of the tree alternate between MAX and MIN players
- MAX nodes: try to maximize the score
- MIN nodes: try to minimize the score

Utility (Evaluation) Function

At terminal states, a utility function assigns a numeric value indicating the outcome of the game. Typical utility values:

- Win for MAX: +1 (or positive value)
- Loss for MAX: −1 (or negative value)
- Draw: 0

Activity 3 (Problem Statements / Tasks):

1. *CSV Student Performance Analyzer*

   You are given a CSV file named students.csv. Write a Python program that:

- Reads the CSV file.
- Remove the header row.
- For each student:
  - Convert marks to float.
  - Handle invalid values using try/except.
- Calculate:
  - Total marks
  - Average marks
- Display:
  - Student name
  - Total marks
  - Average marks
- Raise a custom exception and use try/except if:
  - Any student has negative marks.
  - File not found

2. *Inventory Management System*

Design a simple Inventory Management System using OOP and store product data in a JSON file.

Requirements

Part A – OOP Design

- Create a class Product with:
  - Private variable: __quantity
  - Instance variables:
    - name
    - price
  - Methods:
    - add_stock(amount)
    - sell_stock(amount)
    - get_quantity()
  - Rules:
    - Stock cannot go negative.
    - Price must be positive.
- Create a class Inventory
  - Instance variable:
    - products (dictionary storing Product objects)
  - Methods:
    - add_product(name, price, quantity)
    - sell_product(name, quantity)
    - restock_product(name, quantity)
    - display_inventory()

Part B – JSON Storage

Use a given file named: inventory_data.json. Program should:

- Load existing product data from JSON (if file exists).
- Convert JSON data into Product objects.
- Allow user to:
  - Add new product
  - Sell product
  - Restock product

- View inventory
- Save updated data back to JSON file.

Error Handling

Your program must handle:

- File not found (create new file)
- Invalid numeric input
- Selling more stock than available
- Negative quantity
- Invalid product name

3. *Implement Greedy and A* Search*

Given the following weighted graph and heuristic values:

```
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('F', 1)],
    'D': [],
    'E': [],
    'F': []
}

heuristic = {
    'A': 7,
    'B': 6,
    'C': 2,
    'D': 1,
    'E': 3,
    'F': 0
}
```

Requirements

- Implement:
  - Greedy Best First Search
  - A* Search
- Find path from A to F.
- Display:
  - Path found by Greedy
  - Path found by A*
- Compare results and explain:
  - Which algorithm found better path?
  - Why?